RECONOCIMIENTO DE FIGURAS GEOMETRICAS MEDIANTE ESPACIOS DE BUSQUEDA

Documento de Formalización Teórico-Práctica y Algorítmica

SECCIÓN 1: FORMULACION FORMAL COMO PROBLEMA DE BUSQUEDA

El procesamiento de imágenes e interpretación visual se puede abstraer formalmente como la exploración de un espacio de estados discretos dentro de un árbol o grafo de decisión. La meta es transformar una representación sensorial continua e inestructurada en una etiqueta categórica estructurada.

Formulación de los Elementos del Espacio de Búsqueda

- **(a) Estado Inicial (S_0):** Imagen cruda en matriz de píxeles sin procesar $I \in \mathbb{R}^{M \times N}$. Representa la información matemática bruta capturada por el sensor óptico.
- **(b) Espacio de Estados (S):** Conjunto de interpretaciones parciales de la imagen. Un estado S_k se define como una tupla de atributos extraídos $S_k = (L, V, A, C)$, donde L es el número de lados detectados, V los vértices, A los ángulos internos y C el coeficiente de circularidad/curvatura.
- **(c) Conjunto de Acciones (A):** Funciones de extracción e inferencia de rasgos primarios:
  - a_1: Umbralización y segmentación de contornos.
  - a_2: Detección y conteo de vértices mediante aproximación poligonal (Ramer-Douglas-Peucker).
  - a_3: Cálculo de ángulos entre segmentos adyacentes.
  - a_4: Medición de compacidad y relaciones entre área y perímetro.
- **(d) Prueba de Objetivo ($G(S)$):** Función booleana que retorna *Verdadero* cuando la firma geométrica coincida rigurosamente con una de las 5 clases geométricas objetivo (Círculo, Cuadro, Triángulo, Estrella, Trapecio).
- **(e) Función de Costo ($C(S, a, S')$):** Definida como el número total de comparaciones de características y operaciones computacionales acumuladas desde el estado inicial hasta el estado evaluado.

Justificación de la Búsqueda y Selección del Tipo de Búsqueda

¿Por qué es un problema de búsqueda? Es un problema de búsqueda porque existe un espacio infinito de posibles interpretaciones visuales debido al ruido, oclusión, orientación y escala. El algoritmo debe navegar sistemáticamente a través de hipótesis parciales aplicándoles operadores de extracción de rasgos hasta encontrar una coincidencia taxonómica con los patrones conocidos.

Búsqueda Heurística vs. Búsqueda Ciega: La **búsqueda heurística** es considerablemente más adecuada. La búsqueda ciega (no informada) expandiría arbitrariamente todas las combinaciones de características geométricas sin un criterio de ordenamiento, generando una explosión combinatoria ineficiente. La búsqueda heurística utiliza información del dominio (como el número de bordes o compacidad) para guiar el proceso evaluando primero las hipótesis con mayor similitud topológica, reduciendo drásticamente el espacio explorado.

SECCIÓN 2: GRAFO REPRESENTATIVO DEL SISTEMA DE CLASIFICACION

El siguiente grafo ponderado representa las relaciones de similitud entre las figuras objetivo y el estado nulo (*NULL*). Los pesos en las aristas representan el grado de discrepancia morfológica entre clases (a menor costo/distancia, mayor similitud morfológica).

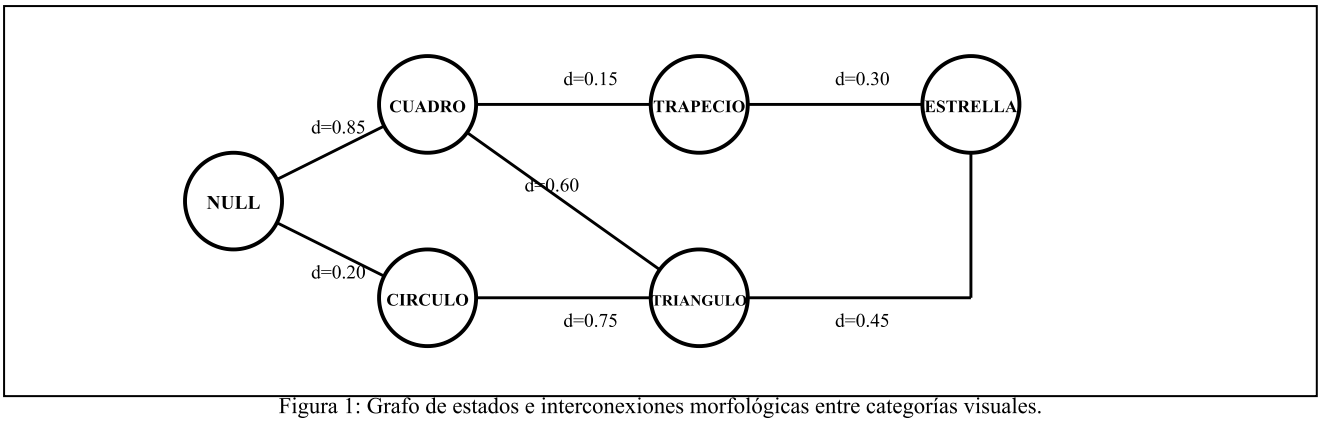


Figura 1: Grafo de estados e interconexiones morfológicas entre categorías visuales.

SECCIÓN 3: ALGORITMOS DE BUSQUEDA APLICADOS

(a) Búsqueda Ciega en Anchura (BFS - Breadth-First Search)

BFS explora el grafo nivel por nivel desde el nodo raíz (*NULL*) utilizando una cola FIFO. Dado que no posee información del dominio, examina exhaustivamente todas las interpretaciones de igual complejidad antes de descender a niveles más profundos.

Flujo de Recorrido: Comienza en *NULL*, luego evalúa los vecinos directos de primer nivel (p. ej., CÍRCULO y CUADRO). Si ninguno satisface los criterios exactos de la imagen de entrada, pasa al siguiente nivel (TRAPECIO, TRIÁNGULO) y finalmente a figuras complejas como ESTRELLA. Garantiza encontrar la solución si existe, pero su complejidad temporal y espacial es $O(b^d)$.

(b) Búsqueda Heurística por Coincidencia de Rasgos

La búsqueda heurística emplea una función de evaluación $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo de extracción acumulado y $h(n)$ es la estimación del error entre los rasgos observados y los teóricos.

Definimos la heurística como el valor absoluto de la diferencia de vértices más la diferencia de compacidad:
$h(n) = | V_{observado} - V_{objetivo} | + | C_{observado} - C_{objetivo} |$

Demostración de Admisibilidad: Una heurística es admisible si nunca sobrestima el costo real para alcanzar la meta ($h(n) \leq h^*(n)$). Dado que nuestra función cuantifica discrepancias geométricas directas normalizadas no negativas sin añadir penalizaciones ficticias o infladas, el valor estimado de diferencia siempre es menor o igual al número real de transformaciones requeridas. Por lo tanto, la heurística es **admisible**, lo que garantiza la optimalidad en algoritmos como A*.

(c) Algoritmo Genético para Clasificación Visual

Los algoritmos genéticos optimizan la búsqueda del vector de parámetros que mejor se ajusta a la figura de la imagen cruda mediante evolución simulada.

- **Cromosoma 1 (Estructura de Indivduo):** Un vector discreto de 4 genes: $[V, A, P, C]$ donde:
  - V: Número de vértices (entero).
  - A: Ángulo medio de vértices (grados).
  - P: Perímetro normalizado (flotante).
  - C: Coeficiente de Compacidad $4\pi \cdot Área / Perímetro^2$.
- **Función de Aptitud (Fitness Function):**
  $Fitness(Cromosoma) = 1 / (1 + \Sigma w_i | Rasgo_{i, obs} - Rasgo_{i, crom} |)$
  Un valor cercano a 1.0 indica coincidencia perfecta con la figura analizada.
- **Operadores Genéticos:**
  - *Cruce (Crossover):* Cruce en un solo punto combinando genes estructurales de dos modelos de figura candidatos.
  - *Mutación:* Alteración aleatoria con probabilidad $p_m = 0.05$ en un gen (ej. variar V en ±1 o modificar A).
- **Criterio de Parada y Flujo Completo:** La población evoluciona durante N generaciones o hasta que el valor de *Fitness* alcance un umbral superior a 0.95. El flujo comprende: Inicialización aleatoria → Evaluación de Aptitud → Selección por Ruleta → Cruce y Mutación → Reemplazo de Población → Identificación de la Figura Ganadora.

SECCIÓN 4: CODIGO HTML E INTEGRACIÓN CON OPENCV.JS

A continuación se presenta la implementación completa en código HTML e integración con la librería OpenCV en JavaScript (OpenCV.js) para detectar y clasificar figuras en tiempo real a partir de una imagen.

```
<!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <title>Reconocimiento de Figur
as con OpenCV.js</title> <script async src="https://docs.opencv.org/4.x/opencv.js" type="text/javas
cript"></script> <style> body { font-family: sans-serif; text-align: center; color: #000; backgroun
d: #fff; } canvas { border: 1px solid #000; margin-top: 10px; } </style> </head> <body> <h2>Clasifi
cador de Figuras Geometricas</h2> <input type="file" id="fileInput" accept="image/*" /> <br/> <canv
as id="canvasOutput"></canvas> <script type="text/javascript"> document.getElementById('fileInpu
t').addEventListener('change', (e) => { let imgElement = document.createElement('img'); imgElement.
src = URL.createObjectURL(e.target.files[0]); imgElement.onload = function() { let src = cv.imread
(imgElement); let gray = new cv.Mat(); let thresh = new cv.Mat(); let contours = new cv.MatVector
(); let hierarchy = new cv.Mat(); // 1. Escala de grises y umbralización cv.cvtColor(src, gray, cv.
COLOR_RGBA2GRAY, 0); cv.threshold(gray, thresh, 120, 255, cv.THRESH_BINARY_INV); // 2. Extracción d
e contornos (Acción de búsqueda) cv.findContours(thresh, contours, hierarchy, cv.RETR_TREE, cv.CHAI
N_APPROX_SIMPLE); for (let i = 0; i < contours.size(); ++i) { let cnt = contours.get(i); let poly =
new cv.Mat(); let peri = cv.arcLength(cnt, true); cv.approxPolyDP(cnt, poly, 0.04 * peri, true); le
t vertices = poly.rows; let shape = "NULL"; // 3. Prueba de objetivo según número de vértices y ras
gos if (vertices === 3) shape = "TRIANGULO"; else if (vertices === 4) { let rect = cv.boundingRect
(poly); let ar = rect.width / parseFloat(rect.height); shape = (ar >= 0.95 && ar <= 1.05) ? "CUADR
O" : "TRAPECIO"; } else if (vertices > 4 && vertices <= 6) shape = "ESTRELLA"; else if (vertices >
6) shape = "CIRCULO"; // Dibujar el resultado cv.drawContours(src, contours, i, new cv.Scalar(0, 0,
0, 255), 2); } cv.imshow('canvasOutput', src); src.delete(); gray.delete(); thresh.delete(); contou
rs.delete(); hierarchy.delete(); }; }); </script> </body> </html>
```